

Module 04

LLM Fundamentals

Large Language Models dengan Transformers



transformer + data masif

Act 1

Apa itu LLM?

Dari NLP ke model bahasa raksasa

Definisi LLM

NLP itu bidang luas, LLM satu jenis model di dalamnya

- ▶ **LLM** = *Large Language Model*, jenis model **NLP** khusus
- ▶ Dilatih pada teks masif (miliaran kata dari internet)
- ▶ Berbasis arsitektur **transformer** + deep learning
- ✓ **NLP**: bidang luas
- ✓ **LLM**: satu jenis model di dalamnya
- ▶ Jago menulis, menerjemahkan, menjawab seperti manusia

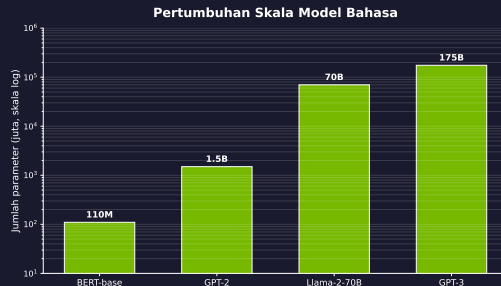


Data masif + **transformer** = pemahaman bahasa luar biasa — lompatan besar di dalam NLP.

Skala & Evolusi

Skala parameter membantu, tapi data & arsitektur juga menentukan

- ▶ Evolusi: **n-gram** → **RNN/LSTM** → **Transformer** (2017)
- ▶ **Transformer** memproses kata secara paralel lewat **attention**, bukan satu per satu seperti RNN
- ▶ Umumnya makin banyak **parameter**, makin mampu menangkap pola (sampai titik tertentu)
- ▶ Kualitas **data**, arsitektur & fine-tuning sama pentingnya



Terobosan **Transformer** 2017 memicu ledakan ukuran: BERT 110M → GPT-3 175B

Aplikasi LLM di Dunia Nyata

Satu model, banyak tugas bahasa

- ✓ **Chatbot** & asisten: ChatGPT, Gemini, customer service
- ✓ **Code generation**: bantu menulis & menjelaskan kode
- ✓ **Translation**: terjemahan antar bahasa
- ✓ **Summarization**: meringkas dokumen panjang
- ✓ **Question answering**: menjawab pertanyaan informatif

Satu **LLM** bisa mengerjakan banyak tugas bahasa sekaligus tanpa dilatih ulang per tugas.

Act 2

Cara Kerja LLM

Apa yang terjadi di balik layar

Tokenization: Teks Menjadi Angka

Langkah pertama: memecah teks jadi token

- ▶ Model tidak baca huruf; ia baca **token**
- ▶ **Token** sering berupa subword (potongan kata)
- ▶ Tiap **token** dipetakan ke sebuah **ID** angka
- ▶ Kata umum = 1 token; kata jarang dipecah jadi beberapa



(ID hanya ilustrasi)

Komputer tidak mengerti kata; semua diubah jadi **token ID** dulu.

Embedding & Transformer

Dari ID angka ke prediksi kata berikutnya



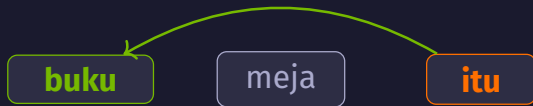
- ▶ Tiap **token ID** → **embedding** (vektor angka); kata mirip makna → vektor mirip
- ▶ Vektor diproses berlapis lewat **transformer block**; output = prediksi **token** berikutnya

Attention secara Intuitif

Model belajar kata mana yang saling merujuk

- ▶ **Attention** = model "memperhatikan" kata paling relevan
- ▶ Saat memproses 1 kata, ia menimbang semua kata lain
- ▶ Model belajar "**itu**" merujuk ke "**buku**", bukan "meja"
- ▶ Kunci agar LLM paham konteks kalimat panjang

"...menaruh buku di meja karena itu berat"



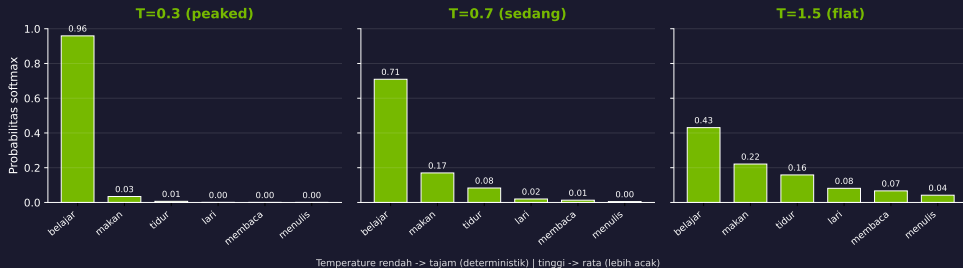
attention: "itu" → "buku"

Attention membuat model fokus pada kata yang benar-benar relevan, bukan semua sama rata.

Generasi Teks & Sampling

Menulis satu token demi satu token

Efek Temperature pada Distribusi Sampling Token



- ▶ LLM **autoregressive**: prediksi 1 **token** lalu ulang
- ▶ Tiap token baru jadi input token selanjutnya

- ▶ **temperature**: rendah = fokus, tinggi = kreatif/acak
- ▶ `do_sample=True` → pilih token acak-terbobot

Act 3

Memakai LLM dengan HuggingFace

Dari teori ke kode

Ekosistem HuggingFace

Pustaka open-source untuk LLM

- ✓ Library **transformers**: API standar memuat & menjalankan model
- ✓ **Model Hub**: ribuan model siap pakai, gratis di-*download*
- ✓ Tiap model punya **model card**: deskripsi, lisensi, cara pakai
- ✓ Model datang lengkap dengan **tokenizer** & **config**

Tidak perlu melatih dari nol — ambil model siap pakai dari Hub.

pipeline vs Load Manual

Dua cara memanggil model

- ▶ **pipeline:** cepat, cukup sebut task + nama model
- ▶ **AutoModel + AutoTokenizer:** manual, lebih fleksibel
- ▶ pipeline cocok untuk eksperimen cepat
- ▶ Load manual: kontrol penuh atas tokenizer & **inference**

```
1 from transformers import pipeline
2
3 pipe = pipeline("text-generation",
4                 model="facebook/opt-350m")
5 pipe("My name is")
```

pipeline = mudah · AutoModel = fleksibel & terkontrol

Model Gated & Login HF

Akses model yang dibatasi

- ▶ Sebagian model **gated**, contoh: **Llama-2** dari Meta
- ▶ Perlu setuju lisensi & punya **token** akses HuggingFace
- ▶ Login dulu lewat **huggingface-cli login**
- ▶ Tujuan: lisensi, kontrol penggunaan, verifikasi pengguna

```
1 !huggingface-cli login
```

token HF → akses model gated

Model gated butuh token: **login** dulu sebelum download.

Text Classification

Mekanik pipeline & pentingnya fine-tuning

- ▶ `pipeline("sentiment-analysis")` di atas **AutoModelForSequenceClassification**
- ▶ **prajjwal1/bert-tiny** baru di-pretrain, **belum** punya head klasifikasi terlatih
- ▶ Akibatnya label = **LABEL_0/LABEL_1**, score ~ 0.5 (tebakan acak)
- ▶ Untuk sentimen nyata: pakai model *fine-tuned* (mis. `distilbert` di **SST-2**)

Teks	Label	Score
I love working with language models!	LABEL_1	0.535
This task is quite challenging.	LABEL_1	0.510
The results are impressive and amazing.	LABEL_1	0.522

Head belum terlatih $\rightarrow$ output ~ 0.5 (acak); butuh model *fine-tuned* untuk sentimen nyata.

Act 4

Aplikasi LLM

Membangun sesuatu yang berguna

Chatbot dengan Gradio

Antarmuka chat dalam beberapa baris kode

- ▶ **gr.ChatInterface** membuat tampilan chat siap pakai
- ▶ Fungsi menyusun **prompt** "User: ... \nAssistant:"
- ▶ Parameter **history** disediakan Gradio (di sini diabaikan)
- ▶ **launch(share=True)** beri tautan publik

```
1 def chatbot_response(message, history):
2     prompt = f"User: {message}\nAssistant:"
3     return generate_text(prompt, model,
4                           tokenizer,
5                           max_length=150)
6
7 iface = gr.ChatInterface(chatbot_response,
8                           title="Simple LLM Chatbot")
9 iface.launch(share=True)
```

Few-shot Learning

Mengajari model lewat contoh di dalam prompt

- ▶ Beri beberapa **contoh** langsung di dalam **prompt**
- ▶ Tanpa melatih ulang, model meniru pola
- ▶ Pola Q/A berulang, pertanyaan baru dibiarkan kosong
- ▶ Prompt lebih panjang → naikan **max_length**

```
1 Question: What is the capital of France?  
2 Answer: The capital of France is Paris.  
3  
4 Question: What is the capital of Japan?  
5 Answer: The capital of Japan is Tokyo.  
6  
7 Question: What is the capital of Italy?  
8 Answer:      <- model melanjutkan
```

Contoh > perintah: tunjukkan pola, model akan menirunya.

Model Evaluation

Mengukur kualitas output secara sederhana

- ▶ Jalankan beberapa **test prompt** ke model
- ▶ Catat tiap **prompt**, **response**, & panjang jawaban
- ▶ Panjang = jumlah kata (metrik sederhana)
- ▶ Dasar untuk metrik lebih lanjut nanti

```
1 for prompt in test_prompts:
2     response = generate_text(prompt,
3                               model,
4                               tokenizer)
5     results.append({
6         'prompt': prompt,
7         'response': response,
8         'length': len(response.split()),
9     })
```

Act 5

Ringkasan & Lanjutan

Apa selanjutnya?

Ringkasan Modul 04

#	Topik	Library/Model	Tujuan
1	LLM inference	transformers (facebook/opt-350m)	Generasi teks dari prompt
2	Pipeline vs manual	pipeline() vs AutoModel (Llama-2-7b, gated)	2 cara load & jalankan model
3	Text classification	prajjwal1/bert-tiny	Klasifikasi sentimen (head ilustratif)
4	Chatbot interface	Gradio (gr.ChatInterface)	UI percakapan sederhana
5	Few-shot learning	facebook/opt-350m	Beri contoh di prompt, model meniru
6	Model evaluation	transformers (fungsi sendiri)	Uji respons & ukur panjang output

Setup, Tools & Lanjut ke Module 05

Model kecil, hemat memori, dan kenapa kita butuh RAG

- ▶ Model kecil: opt-350m, bert-tiny
- ▶ Contoh model besar gated:
Llama-2-7b-chat-hf
- ▶ float16 → bobot ~setengah memori vs float32
- ▶ Butuh GPU: Colab Tesla T4 (!nvidia-smi)

```
1 model = AutoModelForCausalLM\  
2     .from_pretrained(  
3         "facebook/opt-350m",  
4         device_map="auto",  
5         torch_dtype=torch.float16,  
6     )
```

LLM bisa **halusinasi** & punya **knowledge cutoff** → **Module 05 (RAG)** memberi LLM "buku contekan".

Terima Kasih!

Pertanyaan? Diskusi?

LLM Fundamentals · Modul 04
Navasena Gen-ML Course